JETBRAINS | Kotlin

Generics

kotlinglang.org/education



What? Why?

```
fun quickSort(collection: CollectionOfInts) { ... }
quickSort(listOf(1, 2, 3)) // OK
quickSort(listOf(1.0, 2.0, 3.0)) // NOT OK

fun quickSort(collection: CollectionOfDoubles)
{ ... } // overload (we'll get back to this a bit later)
quickSort(listOf(1.0, 2.0, 3.0)) // OK
quickSort(listOf(1, 2, 3)) // OK
```

Kotlin Number inheritors: Int, Double, Byte, Float, Long, Short Do we need 4 more implementations of quickSort?

How?

Does the quickSort algorithm actually care what is it sorting? No, as long as it can compare two values against each other.

```
fun <T : Comparable<T>> quickSort(collection:
Collection<T>): Collection<T> { ... }

quickSort(listOf(1.0, 2.0, 3.0)) // OK

quickSort(listOf(1, 2, 3)) // OK

quickSort(listOf("one", "two", "three")) // OK
```

How?

Generics allow you to write code that can work with any type or with types that should satisfy some rules (constraints) but are not limited in any other ways: type parameters.

```
class Holder<T>(val value: T) { ... }

val intHolder = Holder<Int>(23)

val cupHolder = Holder("cup") // Generic parameter type can be inferred
```

Constraints

Sometimes we do not want to work with an arbitrary type and expect it to provide us with some functionality. In such cases type constraints in the form of upper bounds are used: upper bounds.

```
class Pilot<T : Movable>(val vehicle: T)
{
    fun go() { vehicle.move() }
}

val ryanGosling = Pilot<Car>(Car("Chevy",
"Malibu"))
val sullySullenberger =
Pilot<Plane>(Plane("Airbus", "A320"))
```

Constraints continued

There can be several parameter types, and generic classes can participate in inheritance. `public interface MutableMap<K, V> : Map<K, V> { ... }`

There can also be several constraints (which means the type parameter has to implement several interfaces): `fun <T, S> moveInAnAwesomeWayAndCompare(a: T, b: S) where T : Comparable<T>, S : Comparable<T>, T : Awesome, T : Movable { ... }`

Star-projection

When you do not care about the parameter type, you can use *star-projection* `*` (`Any? / Nothing`).

```
fun printKeys(map: MutableMap<*, *>)
{ ... }
```

Let's go back

```
open class A
open class B : A()
class C : B()

Nothing <: C <: B <: A <: Any
```

This means that the `Any` class is the *superclass* for all the classes and at the same time `Nothing` is a *subtype* of any type

What is next?

Consider a basic example:

```
interface Holder<T> {
    fun push(newValue: T) // consumes an
element

}

fun pop(): T // produces an element

fun size(): Int // does not interact with
T
```

What is next?

```
interface Holder<T> {
     fun push(newValue: T) // consumes an element

fun pop(): T // produces an element

}

fun size(): Int // does not interact with T
```

In Kotlin there are type projections:

```
G<T> // invariant, can consume and produce elements
G<in T> // contravariant, can only consume elements
G<out T> // covariant, can only produce elements
G<*> // star-projection, does not interact with T
```

Several examples

```
G<T> // invariant, can consume and produce
elements

interface Holder<T> {
     fun push(newValue: T)// consumes an
element: OK

}

fun pop(): T // produces an element: OK

fun size(): Int // does not interact with T:
OK
```

Several examples

```
G<in T> // contravariant, can only consume
elements

interface Holder<in T> {
     fun push(newValue: T) // consumes an
element: OK

     fun pop(): T // produces an element:
ERROR: [TYPE_VARIANCE_CONFLICT_ERROR] Type
parameter T is declared as 'in' but occurs in
'out' position in type T

}

fun size(): Int // does not interact with T:
OK
```

Several examples

```
G<out T> // covariant, can only produce
elements

interface Holder<out T> {
     fun push(newValue: T) // consumes an
element: ERROR:
[TYPE_VARIANCE_CONFLICT_ERROR] Type parameter
T is declared as 'out' but occurs in
'in' position in type T

}

fun pop(): T // produces an element: OK

fun size(): Int // does not interact with T:
OK
```

Several examples

```
interface Holder<T> {
     fun push(newValue: T) // consumes an element: OK
     fun pop(): T // produces an element: OK
     fun size(): Int // does not interact with T: OK
}

fun <T> foo1(holder: Holder<T>, t: T) {
     holder.push(t) // OK
                                                fun
foo1(holder: Holder<Any>, t: Any) {
}

holder.push(t) // OK
fun <T> foo2(holder: Holder<*>, t: T) {        }
     holder.push(t) // ERROR: [TYPE_MISMATCH] Type
mismatch. Required: Nothing. Found: T
}
```

Subtyping

```
open class A
open class B : A()      ----->  Nothing <: C <: B <: A
<: Any
class C : B()

class Holder<T>(val value: T) { ... }

Holder<Nothing> ??? Holder<C> ??? Holder<B> ???
Holder<A> ??? Holder<Any>
```

Subtyping

```
open class A
open class B : A()      ----->  Nothing <: C <: B <: A
<: Any
class C : B()

class Holder<T>(val value: T) { ... }
```

`Holder<Nothing>` ~~<:~~ `Holder<C>` ~~<:~~ `Holder<B>` ~~<:~~ `Holder<A>` ~~<:~~ `Holder<Any>`

~~<:<:<:~~

Generics are invariant!!

```
val c: C = C()
val b: B = c // C <: B, OK
```

VS

```
val holderC: Holder<C> = Holder(C())
val holderB: Holder<B> = holderC // ERROR: Type
mismatch.
Required: Holder<B>. Found: Holder<C>.
```

Subtyping

```
open class A
open class B : A()      ----->  Nothing <: C <: B <: A
<: Any
class C : B()

class Holder<T>(val value: T) { ... }

val holderC: Holder<C> = Holder(C())
val holderB: Holder<B> = holderC //ERROR: Type
mismatch. Required: Holder<B>. Found: Holder<C>.
```

BUT

```
val holderB: Holder<B> = Holder(C()) //
OK, because of casting
```

Subtyping

```
class Holder<T> (var value: T?) {
    fun pop(): T? = value.also { value = null }
    fun push(newValue: T?): T? = value.also { value =
newValue }
    fun steal(other: Holder<T>) { value = other.pop() }
    fun gift(other: Holder<T>) { other.push(pop()) }
}
```

`Holder<Nothing>` <: `Holder<C>` <: `Holder<B>` <: `Holder<A>` <: `Holder<Any>`

<:><:><:>

```
val holderB: Holder<B> = Holder(B())
val holderA: Holder<A> = Holder(null)
holderA.steal(holderB) // ERROR: Type mismatch.
Required: Holder<A>. Found: Holder<B>.
holderB.gift(holderA) // ERROR: Type mismatch.
Required: Holder<B>. Found: Holder<A>.
```

Type projection: in

```
class Holder<T> (var value: T?) {
    ...
    fun gift(other: Holder<in T>) { other.push(pop()) }
}
holderB.gift(holderA) // OK
```

Type projection: `other` is a restricted (projected) generic. You can only call methods that **accept** the type parameter `T`, which in this case means that you can only call `push()`.

This is contravariance:

```
Nothing <: C <: B <: A <: Any
Holder<Nothing> :> Holder<C> :> Holder<B> :>
Holder<A> :> Holder<Any>
```

Type projection: out

```
class Holder<T> (var value: T?) {
    ...
    fun steal(other: Holder<out T>) { value =
other.pop() }
}
holderA.steal(holderB) // OK
```

Type projection: `other` is a restricted (projected) generic. You can only call methods that **return** the type parameter `T`, which in this case means that you can only call `pop()`.

This is covariance:
`Nothing <: C <: B <: A <: Any`
`Holder<Nothing> <: Holder<C> <: Holder<B> <: Holder<A> <: Holder<Any>`

Type projections

```
class Holder<T> (var value: T?) {
   fun steal(other: Holder<out T>) {
      val oldValue = push(other.pop())
      other.push(oldValue) // ERROR: Type mismatch.
Required: Nothing?. Found: T?.
   }
   fun gift(other: Holder<in T>) {
      val otherValue = other.push(pop())
      push(otherValue) // ERROR: Type mismatch.
Required: T?. Found: Any?.
   }
}
```

`out T` returns something that can be cast to `T` and accepts literally `Nothing`. `in T` accepts something that can be cast to `T` and returns a meaningless `Any?`.

Type erasure

At runtime, the instances of generic types do not hold any information about their actual type arguments. The type information is said to be erased. The same byte-code is used in all usages of the generic as opposed to C++, where each template is compiled separately for each type parameter provided.

- Any `MutableMap<K, V>` becomes `MutableMap<*, *>` in the runtime*.
- Any `Pilot<T : Movable>` becomes `Pilot<Movable>`.

* Actually, in the **Kotlin/JVM** runtime we have just `java.util.Map` to preserve compatibility with Java.

Type erasure

As a corollary, you cannot override a function **(in Kotlin/JVM)** by changing generic type parameters:

```
fun quickSort(collection: Collection<Int>) { ... }
fun quickSort(collection: Collection<Double>) { ... }
```

Both become `quickSort(collection: Collection<*>)` and their signatures clash.

But you can use the `JvmName` annotation:

```
@JvmName("quickSortInt")
fun quickSort(collection: Collection<Int>) { ... }
fun quickSort(collection: Collection<Double>) { ... }
```

Nullability in generics

Contrary to common sense, in Kotlin a type parameter specified as `T` can be nullable. `class Holder<T>(val value: T) { ... } // Notice there is no `?` val holderA: Holder<A?> = Holder(null) // T = A? and that is OK`

To prohibit such behavior, you can use a non-nullable `Any` as a constraint.

```
class Holder<T : Any>(val value: T) { ... }
val holderA: Holder<A?> = Holder(null) // ERROR: Type argument is not within its bounds.
Expected: Any. Found: A?.
```

You may also find intersection helpful:

```
fun <T> elvisLike(x: T, y: T & Any): T & Any = x ?: y
```

`T & Any` is populated with all values from `T` besides `null`

Inline functions

If they are used as first-class objects, functions are stored as objects, thus requiring memory allocations, which introduce runtime overhead.

```
fun foo(str: String, call: (String) → Unit) {
     call(str)
}

fun main() {
     foo("Top level function with lambda example") {
print(it) }
}
```

Inline functions

```
fun foo(str: String, call: (String) → Unit) {
    call(str)
}
```



```
public static final void foo(@NotNull String str,
@NotNull Function1 call) {
    Intrinsics.checkNotNullParameter(str, "str");
    Intrinsics.checkNotNullParameter(call, "call");
    call.invoke(str);
}
public static final void main() {
    foo("Top level function with lambda example",
(Function1)foo$call$lambda$1.INSTANCE);
}
```

This `call` invokes the `print` function by passing the `string` as an argument.

Inline functions

```
public static final void foo(@NotNull String str,
@NotNull Function1 call) {
     Intrinsics.checkNotNullParameter(str, "str");
     Intrinsics.checkNotNullParameter(call, "call");
     call.invoke(str);
}
```

“Under the hood” an instance of a Function class is created, i.e. allocated:

```
foo("...", new Function() {
     @Override
     public void invoke() {
           ...
     }
});
```

Inline functions

We can use the `inline` keyword to inline the function, copying its code to the call site:

```
inline fun foo(str: String, call: (String) → Unit) {
    call(str)
}
fun main() {
    foo("Top level function with lambda example", ::print)
}
```



```
public static final void main() {
    String str$iv = "Top level function with lambda example";
    int $i$f$foo = false;
    int var3 = false;
    System.out.print(str$iv);
}
```

Inline functions

`inline` affects not only the function itself, but also all the lambdas passed as arguments. If you do not want some of the lambdas passed to an `inline` function to be inlined (*for example, inlining large functions is not recommended*), you can mark some of the function parameters with the `noinline` modifier.

```
inline fun foo(str: String, call1: (String) → Unit,
noinline call2: (String) → Unit) {
     call1(str) // Will be inlined
     call2(str) // Will not be inlined
}
```

Inline functions

You can use `return` in inlined lambdas, this is called *non-local return*, which can lead to unexpected behaviour:

```
inline fun foo(call1: () → Unit, call2: () → Unit) {
     call1()
     call2()
}
fun main() {
     println("Step#1")
     foo({ println("Step#2")
           return },
         { println("Step#3") })
     println("Step#4")
}

→ Output:
     Step#1
     Step#2
```

Inline functions

To prohibit returning from the lambda expression we can mark the lambda as `crossinline`.

```
inline fun foo(crossinline call1: () → Unit, call2:
() → Unit) {
     call1()
     call2()
}
fun main() {
     println("Step#1")
     foo({ println("Step#2")
           return }, //
ERROR: 'return' is not allowed here
          { println("Step#3") })
     println("Step#4")
}
```

`return@foo` is allowed and

fine, though

Inline functions

`crossinline` is especially useful when the lambda from an `inline` function is being called from another context, for example, if it is used to instantiate a `Runnable`:

```
inline fun drive(crossinline specialCall: (String) → Unit, call: (String) → Unit) {
   val nightCall = Runnable { specialCall("There's something inside you") }
   call("I'm giving you a nightcall to tell you how I feel")
   thread { nightCall.run() }
   call("I'm gonna drive you through the night, down the hills")
}
fun main() {
   drive({ System.err.println(it) }) { println(it) }
}
```

Inline reified functions

Sometimes you need to access a type passed as a parameter:

```
fun <T: Animal> foo() {
     println(T::class) // ERROR: Cannot use 'T' as
reified type parameter. Use a class instead
——→ add a param: t: KClass<T>
}
```

You can use the `reified` keyword with `inline` functions:

```
inline fun <reified T: Animal> foo() {
     println(T::class) // OK
}
```

Note that the compiler has to be able to know the actual type passed as a type argument so that it can modify the generated bytecode to use the corresponding class directly.

```
open class A
class B : A()
class C : A() { fun consume(other: A): C = this }


fun <T, S : R, R> funny(
   source: Iterator<????>,
   target: MutableCollection<????>,
   base: ????,
   how: ????
) {
   var result: R = base
   for (value in source) {
       result = how(result, value)
       target.add(result)
   }
}


fun main() {
   val wtf = mutableListOf<A>()
   val src = mapOf(3.14 to B(), 2 to B(), "Hello" to B())
   val c = C()
   funny(src.values.iterator(), wtf, c) { r, t → r.consume(t) }
}
```

(in)Variance

```
class Holder<T>(val value: T) { ... }

open class A

open class B : A()

class C : B()
```

A hierarchy is in place (and don't forget about the same hierarchy with nullability):

```
Nothing → C → B → A → Any
```

Variance of Generics would give us another hierarchy:

```
Holder<Nothing> → Holder<C> → Holder<B> → Holder<A> → Holder<Any>
```

But this is not the case, since Generics are invariant.

```
val holderC = Holder(C())

val holderB: Holder<B> = holderC // Error: Type mismatch. Required:
Holder<B>. Found: Holder<C>.
```

NB: code below works, since `C()` passed as an argument is being cast to `B`, nothing to do with variance. `val kotlinIsSmart: Holder<B> = Holder(C())`

More

```
class Holder<T> (var value: T?) {
    fun pop(): T? = value.also { value = null }
    fun push(newValue: T?): T? = value.also { value =
newValue }
    fun steal(other: Holder<T>) { value = other.pop() }
    fun gift(other: Holder<T>) { other.push(pop()) }
}
val holderB: Holder<B> = Holder(B())
val holderA: Holder<A> = Holder(null)
holderA.steal(holderB) // Error: Type mismatch.
Required: Holder<A>. Found: Holder<B>.
holderB.gift(holderA) // Error: Type mismatch.
Required: Holder<B>. Found: Holder<A>.
```

But why not? `B` can be easily cast to `A` inside `steal` or `gift` and everything should be fine.

JETBRAINS | Kotlin

Thank you

